Homework 6

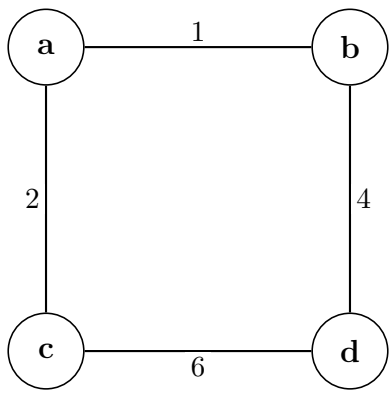
Yousef Alaa Awad

April 13, 2026

Lab 1: DV Routing Algorithm

Task 1-1: DV Routing for a 2×2 Node Network

The 2×2 network contains 4 nodes (a, b, c, d) connected as a grid:



The DV algorithm converges in **3 iterations**. Initial DV tables contain only direct neighbor costs (non-neighbors set to ∞). With each iteration, nodes exchange distance vectors with neighbors and update using the Bellman-Ford equation: $D_x(y) = \min_v\{c(x,v) + D_v(y)\}$.

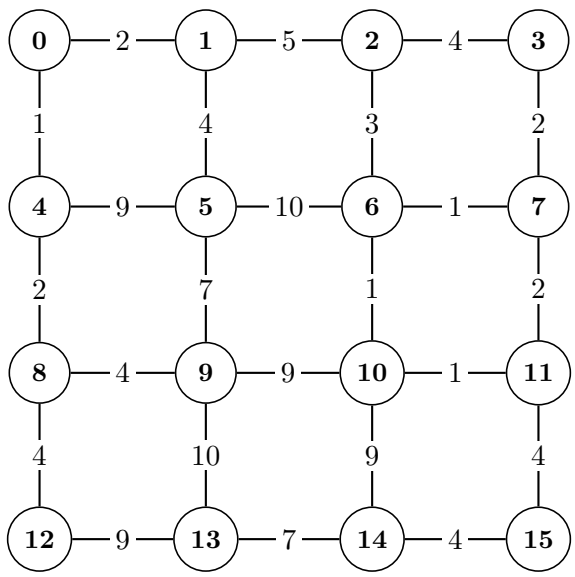
Listing 1: Task 1-1 Output

```
====================================================================
  TASK 1-1: DV Routing for 2x2 (4-node) Network
====================================================================

  Network topology:
    a -- b : cost 1
    a -- c : cost 2
    b -- d : cost 4
    c -- d : cost 6

============================================================
  Task 1-1 - Iteration 0
============================================================

  DV table at node a:
            a     b     c     d
       a    0     1     2   inf

  DV table at node b:
            a     b     c     d
       b    1     0   inf     4

  DV table at node c:
```

```
            a     b     c     d
      c     2   inf     0     6

  DV table at node d:
            a     b     c     d
      d   inf     4     6     0

============================================================
  Task 1-1 - Iteration 1
============================================================

  DV table at node a:
            a     b     c     d
      a     0     1     2     5
```

Task 1-2: DV Routing for a 4×4 Node Network

The 4×4 grid network has 16 nodes (0–15) with randomly generated edge weights in $[1, 10]$:



The DV algorithm converges in **7 iterations** (with seed=42).

Listing 2: Task 1-2 Output

```
====================================================================
  TASK 1-1: DV Routing for 2x2 (4-node) Network
====================================================================

  Network topology:
    a -- b : cost 1
    a -- c : cost 2
    b -- d : cost 4
    c -- d : cost 6

============================================================
  Task 1-1 - Iteration 0
============================================================

  DV table at node a:
            a     b     c     d
      a     0     1     2   inf

  DV table at node b:
            a     b     c     d
      b     1     0   inf     4

  DV table at node c:
            a     b     c     d
      c     2   inf     0     6

  DV table at node d:
            a     b     c     d
```

```
      d   inf     4     6     0

============================================================
  Task 1-1 - Iteration 1
============================================================

  DV table at node a:
            a     b     c     d
      a     0     1     2     5

  DV table at node b:
            a     b     c     d
      b     1     0     3     4

  DV table at node c:
            a     b     c     d
      c     2     3     0     6

  DV table at node d:
            a     b     c     d
      d     5     4     6     0

============================================================
  Task 1-1 - Iteration 2
============================================================

  DV table at node a:
            a     b     c     d
      a     0     1     2     5

  DV table at node b:
            a     b     c     d
      b     1     0     3     4

  DV table at node c:
            a     b     c     d
      c     2     3     0     6

  DV table at node d:
            a     b     c     d
      d     5     4     6     0

  >> Converged after 2 iteration(s).


====================================================================
  TASK 1-2: DV Routing for 4x4 (16-node) Network
====================================================================

  Network topology (random weights, seed=42):
     0 --  1 : cost 2
     0 --  4 : cost 1
     1 --  2 : cost 5
     1 --  5 : cost 4
     2 --  3 : cost 4
     2 --  6 : cost 3
     3 --  7 : cost 2
     4 --  5 : cost 9
     4 --  8 : cost 2
     5 --  6 : cost 10
     5 --  9 : cost 7
     6 --  7 : cost 1
     6 -- 10 : cost 1
     7 -- 11 : cost 2
     8 --  9 : cost 4
     8 -- 12 : cost 4
     9 -- 10 : cost 9
     9 -- 13 : cost 10
    10 -- 11 : cost 1
    10 -- 14 : cost 9
    11 -- 15 : cost 4
    12 -- 13 : cost 9
    13 -- 14 : cost 7
    14 -- 15 : cost 4

============================================================
  Task 1-2 - Iteration 0
============================================================

  DV table at node 0:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      0     0     2   inf   inf     1   inf   inf   inf   inf   inf   inf   inf   inf   inf   inf   inf

  DV table at node 1:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      1     2     0     5   inf   inf     4   inf   inf   inf   inf   inf   inf   inf   inf   inf   inf

  DV table at node 2:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      2   inf     5     0     4   inf   inf     3   inf   inf   inf   inf   inf   inf   inf   inf   inf

  DV table at node 3:
```

```
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      3   inf   inf     4     0   inf   inf   inf     2   inf   inf   inf   inf   inf   inf   inf   inf

  DV table at node 4:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      4     1   inf   inf   inf     0     9   inf   inf     2   inf   inf   inf   inf   inf   inf   inf

  DV table at node 5:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      5   inf     4   inf   inf     9     0    10   inf   inf     7   inf   inf   inf   inf   inf   inf

  DV table at node 6:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      6   inf   inf     3   inf   inf    10     0     1   inf   inf     1   inf   inf   inf   inf   inf

  DV table at node 7:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      7   inf   inf   inf     2   inf   inf     1     0   inf   inf   inf     2   inf   inf   inf   inf

  DV table at node 8:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      8   inf   inf   inf   inf     2   inf   inf   inf     0     4   inf   inf     4   inf   inf   inf

  DV table at node 9:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      9   inf   inf   inf   inf   inf     7   inf   inf     4     0     9   inf   inf    10   inf   inf

  DV table at node 10:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     10   inf   inf   inf   inf   inf   inf     1   inf   inf     9     0     1   inf   inf     9   inf

  DV table at node 11:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     11   inf   inf   inf   inf   inf   inf   inf     2   inf   inf     1     0   inf   inf   inf     4

  DV table at node 12:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     12   inf   inf   inf   inf   inf   inf   inf   inf     4   inf   inf   inf     0     9   inf   inf

  DV table at node 13:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     13   inf   inf   inf   inf   inf   inf   inf   inf   inf    10   inf   inf     9     0     7   inf

  DV table at node 14:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     14   inf   inf   inf   inf   inf   inf   inf   inf   inf   inf     9   inf   inf     7     0     4

  DV table at node 15:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     15   inf   inf   inf   inf   inf   inf   inf   inf   inf   inf   inf     4   inf   inf     4     0

============================================================
  Task 1-2 - Iteration 1
============================================================

  DV table at node 0:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      0     0     2     7   inf     1     6   inf   inf     3   inf   inf   inf   inf   inf   inf   inf

  DV table at node 1:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      1     2     0     5     9     3     4     8   inf   inf    11   inf   inf   inf   inf   inf   inf

  DV table at node 2:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      2     7     5     0     4   inf     9     3     4   inf   inf     4   inf   inf   inf   inf   inf

  DV table at node 3:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      3   inf     9     4     0   inf   inf     3     2   inf   inf   inf     4   inf   inf   inf   inf

  DV table at node 4:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      4     1     3   inf   inf     0     9    19   inf     2     6   inf   inf     6   inf   inf   inf

  DV table at node 5:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      5     6     4     9   inf     9     0    10    11    11     7    11   inf   inf    17   inf   inf

  DV table at node 6:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      6   inf     8     3     3    19    10     0     1   inf    10     1     2   inf   inf    10   inf

  DV table at node 7:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      7   inf   inf     4     2   inf    11     1     0   inf   inf     2     2   inf   inf   inf     6

  DV table at node 8:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      8     3   inf   inf   inf     2    11   inf   inf     0     4    13   inf     4    13   inf   inf

  DV table at node 9:
```

```
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      9   inf    11   inf   inf     6     7    10   inf     4     0     9    10     8    10    17   inf

 DV table at node 10:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     10   inf   inf     4   inf   inf    11     1     2    13     9     0     1   inf    16     9     5

 DV table at node 11:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     11   inf   inf   inf     4   inf   inf     2     2   inf    10     1     0   inf   inf     8     4

 DV table at node 12:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     12   inf   inf   inf   inf     6   inf   inf   inf     4     8   inf   inf     0     9    16   inf

 DV table at node 13:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     13   inf   inf   inf   inf   inf    17   inf   inf    13    10    16   inf     9     0     7    11

 DV table at node 14:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     14   inf   inf   inf   inf   inf   inf    10   inf   inf    17     9     8    16     7     0     4

 DV table at node 15:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     15   inf   inf   inf   inf   inf   inf   inf     6   inf   inf     5     4   inf    11     4     0

============================================================
 Task 1-2 - Iteration 2
============================================================

 DV table at node 0:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      0     0     2     7    11     1     6    10   inf     3     7   inf   inf     7   inf   inf   inf

 DV table at node 1:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      1     2     0     5     9     3     4     8     9     5    11     9   inf   inf    21   inf   inf

 DV table at node 2:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      2     7     5     0     4     8     9     3     4   inf    13     4     5   inf   inf    13   inf

 DV table at node 3:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      3    11     9     4     0   inf    13     3     2   inf   inf     4     4   inf   inf   inf     8

 DV table at node 4:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      4     1     3     8   inf     0     7    19    20     2     6    15   inf     6    15   inf   inf

 DV table at node 5:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      5     6     4     9    13     7     0    10    11    11     7    11    12    15    17    20   inf

 DV table at node 6:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      6    10     8     3     3    19    10     0     1    14    10     1     2   inf    17    10     6

 DV table at node 7:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      7   inf     9     4     2    20    11     1     0   inf    11     2     2   inf   inf    10     6

 DV table at node 8:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      8     3     5   inf   inf     2    11    14   inf     0     4    13    14     4    13    20   inf

 DV table at node 9:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      9     7    11    13   inf     6     7    10    11     4     0     9    10     8    10    17    14

 DV table at node 10:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     10   inf     9     4     4    15    11     1     2    13     9     0     1    17    16     9     5

 DV table at node 11:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     11   inf   inf     5     4   inf    12     2     2    14    10     1     0   inf    15     8     4

 DV table at node 12:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     12     7   inf   inf   inf     6    15   inf   inf     4     8    17   inf     0     9    16    20

 DV table at node 13:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     13   inf    21   inf   inf    15    17    17   inf    13    10    16    15     9     0     7    11

 DV table at node 14:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     14   inf   inf    13   inf   inf    20    10    10    20    17     9     8    16     7     0     4

 DV table at node 15:
```

```
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
    15   inf   inf   inf     8   inf   inf     6     6   inf    14     5     4    20    11     4     0

============================================================
  Task 1-2 - Iteration 3
============================================================

  DV table at node 0:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     0     0     2     7    11     1     6    10    11     3     7    11   inf     7    16   inf   inf

  DV table at node 1:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     1     2     0     5     9     3     4     8     9     5     9     9    10     9    21    18   inf

  DV table at node 2:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     2     7     5     0     4     8     9     3     4    10    13     4     5   inf    20    13     9

  DV table at node 3:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     3    11     9     4     0    12    13     3     2   inf    13     4     4   inf   inf    12     8

  DV table at node 4:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     4     1     3     8    12     0     7    11    20     2     6    15    16     6    15    22   inf

  DV table at node 5:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     5     6     4     9    13     7     0    10    11     9     7    11    12    15    17    20    16

  DV table at node 6:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     6    10     8     3     3    11    10     0     1    14    10     1     2    18    17    10     6

  DV table at node 7:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     7    11     9     4     2    20    11     1     0    15    11     2     2   inf    17    10     6

  DV table at node 8:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     8     3     5    10   inf     2     9    14    15     0     4    13    14     4    13    20    18

  DV table at node 9:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     9     7     9    13    13     6     7    10    11     4     0     9    10     8    10    17    14

  DV table at node 10:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
    10    11     9     4     4    15    11     1     2    13     9     0     1    17    16     9     5

  DV table at node 11:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
    11   inf    10     5     4    16    12     2     2    14    10     1     0    18    15     8     4

  DV table at node 12:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
    12     7     9   inf   inf     6    15    18   inf     4     8    17    18     0     9    16    20

  DV table at node 13:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
    13    16    21    20   inf    15    17    17    17    13    10    16    15     9     0     7    11

  DV table at node 14:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
    14   inf    18    13    12    22    20    10    10    20    17     9     8    16     7     0     4

  DV table at node 15:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
    15   inf   inf     9     8   inf    16     6     6    18    14     5     4    20    11     4     0

============================================================
  Task 1-2 - Iteration 4
============================================================

  DV table at node 0:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     0     0     2     7    11     1     6    10    11     3     7    11    12     7    16    20   inf

  DV table at node 1:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     1     2     0     5     9     3     4     8     9     5     9     9    10     9    18    18    14

  DV table at node 2:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     2     7     5     0     4     8     9     3     4    10    13     4     5    14    20    13     9

  DV table at node 3:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     3    11     9     4     0    12    13     3     2    14    13     4     4   inf    19    12     8

  DV table at node 4:
```

```
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      4     1     3     8    12     0     7    11    12     2     6    12    16     6    15    22    20

  DV table at node 5:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      5     6     4     9    13     7     0    10    11     9     7    11    12    13    17    20    16

  DV table at node 6:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      6    10     8     3     3    11    10     0     1    13    10     1     2    18    17    10     6

  DV table at node 7:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      7    11     9     4     2    12    11     1     0    15    11     2     2    19    17    10     6

  DV table at node 8:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      8     3     5    10    14     2     9    13    15     0     4    13    14     4    13    20    18

  DV table at node 9:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      9     7     9    13    13     6     7    10    11     4     0     9    10     8    10    17    14

  DV table at node 10:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     10    11     9     4     4    12    11     1     2    13     9     0     1    17    16     9     5

  DV table at node 11:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     11    12    10     5     4    16    12     2     2    14    10     1     0    18    15     8     4

  DV table at node 12:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     12     7     9    14   inf     6    13    18    19     4     8    17    18     0     9    16    20

  DV table at node 13:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     13    16    18    20    19    15    17    17    17    13    10    16    15     9     0     7    11

  DV table at node 14:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     14    20    18    13    12    22    20    10    10    20    17     9     8    16     7     0     4

  DV table at node 15:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     15   inf    14     9     8    20    16     6     6    18    14     5     4    20    11     4     0

============================================================
  Task 1-2 - Iteration 5
============================================================

  DV table at node 0:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      0     0     2     7    11     1     6    10    11     3     7    11    12     7    16    20    16

  DV table at node 1:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      1     2     0     5     9     3     4     8     9     5     9     9    10     9    18    18    14

  DV table at node 2:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      2     7     5     0     4     8     9     3     4    10    13     4     5    14    20    13     9

  DV table at node 3:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      3    11     9     4     0    12    13     3     2    14    13     4     4    18    19    12     8

  DV table at node 4:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      4     1     3     8    12     0     7    11    12     2     6    12    13     6    15    21    20

  DV table at node 5:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      5     6     4     9    13     7     0    10    11     9     7    11    12    13    17    20    16

  DV table at node 6:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      6    10     8     3     3    11    10     0     1    13    10     1     2    17    17    10     6

  DV table at node 7:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      7    11     9     4     2    12    11     1     0    14    11     2     2    19    17    10     6

  DV table at node 8:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      8     3     5    10    14     2     9    13    14     0     4    13    14     4    13    20    18

  DV table at node 9:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      9     7     9    13    13     6     7    10    11     4     0     9    10     8    10    17    14

  DV table at node 10:
```

```
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      10    11     9     4     4    12    11     1     2    13     9     0     1    17    16     9     5

  DV table at node 11:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      11    12    10     5     4    13    12     2     2    14    10     1     0    18    15     8     4

  DV table at node 12:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      12     7     9    14    18     6    13    17    19     4     8    17    18     0     9    16    20

  DV table at node 13:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      13    16    18    20    19    15    17    17    17    13    10    16    15     9     0     7    11

  DV table at node 14:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      14    20    18    13    12    21    20    10    10    20    17     9     8    16     7     0     4

  DV table at node 15:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      15    16    14     9     8    20    16     6     6    18    14     5     4    20    11     4     0

============================================================
  Task 1-2 - Iteration 6
============================================================

  DV table at node 0:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
       0     0     2     7    11     1     6    10    11     3     7    11    12     7    16    20    16

  DV table at node 1:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
       1     2     0     5     9     3     4     8     9     5     9     9    10     9    18    18    14

  DV table at node 2:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
       2     7     5     0     4     8     9     3     4    10    13     4     5    14    20    13     9

  DV table at node 3:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
       3    11     9     4     0    12    13     3     2    14    13     4     4    18    19    12     8

  DV table at node 4:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
       4     1     3     8    12     0     7    11    12     2     6    12    13     6    15    21    17

  DV table at node 5:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
       5     6     4     9    13     7     0    10    11     9     7    11    12    13    17    20    16

  DV table at node 6:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
       6    10     8     3     3    11    10     0     1    13    10     1     2    17    17    10     6

  DV table at node 7:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
       7    11     9     4     2    12    11     1     0    14    11     2     2    18    17    10     6

  DV table at node 8:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
       8     3     5    10    14     2     9    13    14     0     4    13    14     4    13    20    18

  DV table at node 9:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
       9     7     9    13    13     6     7    10    11     4     0     9    10     8    10    17    14

  DV table at node 10:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      10    11     9     4     4    12    11     1     2    13     9     0     1    17    16     9     5

  DV table at node 11:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      11    12    10     5     4    13    12     2     2    14    10     1     0    18    15     8     4

  DV table at node 12:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      12     7     9    14    18     6    13    17    18     4     8    17    18     0     9    16    20

  DV table at node 13:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      13    16    18    20    19    15    17    17    17    13    10    16    15     9     0     7    11

  DV table at node 14:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      14    20    18    13    12    21    20    10    10    20    17     9     8    16     7     0     4

  DV table at node 15:
             0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      15    16    14     9     8    17    16     6     6    18    14     5     4    20    11     4     0

============================================================
```

```
  Task 1-2 - Iteration 7
============================================================

  DV table at node 0:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      0     0     2     7    11     1     6    10    11     3     7    11    12     7    16    20    16

  DV table at node 1:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      1     2     0     5     9     3     4     8     9     5     9     9    10     9    18    18    14

  DV table at node 2:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      2     7     5     0     4     8     9     3     4    10    13     4     5    14    20    13     9

  DV table at node 3:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      3    11     9     4     0    12    13     3     2    14    13     4     4    18    19    12     8

  DV table at node 4:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      4     1     3     8    12     0     7    11    12     2     6    12    13     6    15    21    17

  DV table at node 5:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      5     6     4     9    13     7     0    10    11     9     7    11    12    13    17    20    16

  DV table at node 6:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      6    10     8     3     3    11    10     0     1    13    10     1     2    17    17    10     6

  DV table at node 7:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      7    11     9     4     2    12    11     1     0    14    11     2     2    18    17    10     6

  DV table at node 8:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      8     3     5    10    14     2     9    13    14     0     4    13    14     4    13    20    18

  DV table at node 9:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
      9     7     9    13    13     6     7    10    11     4     0     9    10     8    10    17    14

  DV table at node 10:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     10    11     9     4     4    12    11     1     2    13     9     0     1    17    16     9     5

  DV table at node 11:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     11    12    10     5     4    13    12     2     2    14    10     1     0    18    15     8     4

  DV table at node 12:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     12     7     9    14    18     6    13    17    18     4     8    17    18     0     9    16    20

  DV table at node 13:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     13    16    18    20    19    15    17    17    17    13    10    16    15     9     0     7    11

  DV table at node 14:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     14    20    18    13    12    21    20    10    10    20    17     9     8    16     7     0     4

  DV table at node 15:
            0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
     15    16    14     9     8    17    16     6     6    18    14     5     4    20    11     4     0

  >> Converged after 7 iteration(s).


====================================================================
  SUMMARY
====================================================================
  Task 1-1 (2x2): Converged in 2 iteration(s)
  Task 1-2 (4x4): Converged in 7 iteration(s)
```

Questions

Question: For an $N \times N$ network, how many iterations does it require to complete the DV routing?

Answer:

The DV algorithm converges in at most D iterations, where D is the network diameter

in hops. For an $N \times N$ grid, $D = 2(N-1)$. So convergence requires at most $\mathbf{2(N-1)}$ iterations (plus one to confirm stability). For a 2×2 grid: 2–3 iterations. For a 4×4 grid: 6–7 iterations.

Question: In Figure 2, what could be the potential impact if one specific link becomes disconnected (i.e., its distance becomes infinite)?

Answer:

Adjacent nodes set the link cost to ∞ and trigger reconvergence. If the failed link was on a shortest path, the **count-to-infinity problem** may occur—nodes slowly increment distance estimates before stabilizing. Traffic reroutes through longer alternative paths for increased latency. A single link failure cannot partition a grid (alternative paths always exist), but multiple failures could.

Lab 2: Protection Strategies

Task 2-1: CRC-8 and SEC-DED Encoding

For the input value `0xFA7816EB87CC0911` (64 bits):

- **CRC-8** (polynomial 0x107: x^8+x^2+x+1): The computed 8-bit CRC is `0x5B` (binary: `01011011`). The encoded output is `0xFA7816EB87CC09115B` (72 bits).
- **SEC-DED** (Hamming code + overall parity): 7 Hamming parity bits + 1 overall parity bit = 8 parity bits, producing a 72-bit codeword. The encoded output is `0x17D3605B570F981211`.

Listing 3: Task 2-1 Output

```
======================================================================
  TASK 2-1: CRC-8 and SEC-DED Encoding
======================================================================

  Input value: 0xFA7816EB87CC0911
  Input (binary): 1111101001111000000101101110101110000111110011000000100100010001

  --- CRC-8 (Polynomial 0x107: x^8 + x^2 + x + 1) ---
  CRC-8 value:    0x5B (binary: 01011011)
  Encoded data:   0xFA7816EB87CC09115B
  Encoded (bin):  111110100111100000010110111010111000011111001100000010010001000101011011
  Verification:   OK (remainder = 0)

  --- SEC-DED (Hamming + overall parity) ---
  Total bits:     72 (64 data + 8 parity)
  Encoded value:  0x17D3605B570F981211
  Encoded (bin):  000101111101001101100000010110110101011100001111100110000001001000010001
  Verification:   OK (no error detected)
```

Task 2-2: Error Detection Evaluation

Each error type was injected 1,000,000 times, and the detection rates for CRC-8 and SEC-DED are summarized below:

Error Type	**CRC Rate (%)**	**SEC-DED Rate (%)**
A single-bit error	100.0000	100.0000
Two random-bit errors	100.0000	100.0000
Three random-bit errors	100.0000	100.0000
An 8-bit burst error	100.0000	∼97.61
A 16-bit burst error	∼99.61	∼98.62

Observations:

- CRC-8 detects 100% of burst errors up to 8 bits long (matching the CRC degree), and ∼99.6% of 16-bit bursts.
- SEC-DED detects 100% of 1, 2, and 3-bit errors (by design, it corrects single errors and detects double errors; triple errors are also detected in practice due to the overall parity bit). Burst errors are less reliably detected.

Questions

Question: What is the concept of Hamming distance? Why cannot SEC-DED correct two errors?

Answer:

Hamming distance is the number of bit positions in which two codewords differ. The minimum Hamming distance $d_{\min}$ of a code determines its capabilities: detecting t errors requires $d_{\min} \geq t + 1$; correcting t errors requires $d_{\min} \geq 2t + 1$.

SEC-DED has $d_{\min} = 4$, so it can correct 1 error and detect up to 2. It **cannot correct two errors** because a word with 2 errors may be equidistant from two valid codewords, making it impossible to determine the original.

Question: Typically, SEC-DED is implemented using combinational circuits, while CRC is implemented using sequential circuits. Is it possible to implement CRC using combinational circuits?

Answer:

Yes. CRC is polynomial division in GF(2) using only XOR and shifts. For a fixed-length input, this can be unrolled into a combinational XOR-gate tree. However, it requires many gates, has long propagation delay, and is hardwired to one input length—unlike a sequential LFSR, which is compact and handles arbitrary-length data. Combinational CRC is sometimes used in high-throughput, fixed-frame applications.

Source Code

Lab 1: DV Routing (dv_routing.py)

```python
#!/usr/bin/env python3
"""
Distance Vector (DV) Routing Algorithm Implementation
HW6 - Computer Communications & Networks
Author: Yousef Alaa Awad

Implements the Bellman-Ford based DV routing algorithm for:
  Task 1-1: A 2x2 (4-node) network
  Task 1-2: A 4x4 (16-node) network with random edge weights
"""

import random
import math
import copy

INF = math.inf


def build_2x2_network():
    """
    Build the 2x2 grid network from Figure 1.
    Nodes: a, b, c, d arranged as:
        a --- b
        |     |
        c --- d
    Edge weights chosen as representative values.
    """
    nodes = ['a', 'b', 'c', 'd']
    # Adjacency: (node1, node2, cost)
    edges = [
        ('a', 'b', 1),
        ('a', 'c', 2),
        ('b', 'd', 4),
        ('c', 'd', 6),
    ]
    return nodes, edges


def build_4x4_network(seed=42):
    """
    Build a 4x4 grid network (16 nodes) with random edge weights in [1, 10].
    Nodes labeled 0-15, arranged in a grid:
        0  -  1  -  2  -  3
        |     |     |     |
        4  -  5  -  6  -  7
        |     |     |     |
        8  -  9  - 10 - 11
        |     |     |     |
       12 - 13 - 14 - 15
    """
    random.seed(seed)
    N = 4
    nodes = list(range(N * N))
    edges = []
    for r in range(N):
        for c in range(N):
            node = r * N + c
            # Right neighbor
            if c + 1 < N:
                right = r * N + (c + 1)
                cost = random.randint(1, 10)
                edges.append((node, right, cost))
```

```python
            # Down neighbor
            if r + 1 < N:
                down = (r + 1) * N + c
                cost = random.randint(1, 10)
                edges.append((node, down, cost))
    return nodes, edges


def initialize_dv_tables(nodes, edges):
    """
    Initialize distance vector tables for all nodes.
    Each node's DV table is a dict mapping destination -> estimated cost.
    Initially, a node knows: cost to itself = 0, cost to neighbors = link cost,
    cost to all other nodes = infinity.
    Also returns the adjacency list (neighbors of each node).
    """
    # Build adjacency list
    neighbors = {n: {} for n in nodes}
    for u, v, cost in edges:
        neighbors[u][v] = cost
        neighbors[v][u] = cost

    # Initialize DV tables: dv[node][dest] = estimated distance from node to dest
    dv = {}
    for n in nodes:
        dv[n] = {}
        for dest in nodes:
            if dest == n:
                dv[n][dest] = 0
            elif dest in neighbors[n]:
                dv[n][dest] = neighbors[n][dest]
            else:
                dv[n][dest] = INF
    return dv, neighbors


def print_dv_tables(nodes, dv, iteration, label=""):
    """
    Print all DV tables for a given iteration in a readable format.
    Each node's table shows estimated distances to all destinations.
    """
    header = f"{'':>6}" + "".join(f"{str(d):>6}" for d in nodes)

    if label:
        print(f"\n{'='*60}")
        print(f"  {label} - Iteration {iteration}")
        print(f"{'='*60}")
    else:
        print(f"\n--- Iteration {iteration} ---")

    for n in nodes:
        print(f"\n  DV table at node {n}:")
        print(f"  {header}")
        # In a full DV table, node n stores what it believes the distance
        # from itself to every destination is.
        row = f"  {str(n):>6}"
        for dest in nodes:
            val = dv[n][dest]
            if val == INF:
                row += f"{'inf':>6}"
            else:
                row += f"{val:>6}"
        print(row)


def run_dv(nodes, edges, label=""):
    """
    Run the Distance Vector routing algorithm until convergence.
```

```python
    In each iteration, every node sends its distance vector to its neighbors.
    Each neighbor updates its own DV using the Bellman-Ford equation:
        D_x(y) = min over all neighbors v of { c(x,v) + D_v(y) }
    The algorithm terminates when no DV table changes in an iteration.
    """
    dv, neighbors = initialize_dv_tables(nodes, edges)

    # Print initial state (iteration 0)
    print_dv_tables(nodes, dv, 0, label)

    iteration = 0
    while True:
        iteration += 1
        changed = False
        # Create a snapshot of current DV tables so all nodes use
        # the same "round" of information (synchronous update)
        old_dv = copy.deepcopy(dv)

        for n in nodes:
            for dest in nodes:
                if dest == n:
                    continue
                # Bellman-Ford: check all neighbors for a shorter path
                best = old_dv[n][dest]
                for nbr, link_cost in neighbors[n].items():
                    candidate = link_cost + old_dv[nbr][dest]
                    if candidate < best:
                        best = candidate
                if best != dv[n][dest]:
                    dv[n][dest] = best
                    changed = True

        print_dv_tables(nodes, dv, iteration, label)

        if not changed:
            print(f"\n  >> Converged after {iteration} iteration(s).")
            break

    return iteration


def main():
    # ---- Task 1-1: 2x2 Network ----
    print("=" * 70)
    print("  TASK 1-1: DV Routing for 2x2 (4-node) Network")
    print("=" * 70)
    nodes_2x2, edges_2x2 = build_2x2_network()
    print("\n  Network topology:")
    for u, v, c in edges_2x2:
        print(f"    {u} -- {v} : cost {c}")
    iters_2x2 = run_dv(nodes_2x2, edges_2x2, label="Task 1-1")

    # ---- Task 1-2: 4x4 Network ----
    print("\n\n" + "=" * 70)
    print("  TASK 1-2: DV Routing for 4x4 (16-node) Network")
    print("=" * 70)
    nodes_4x4, edges_4x4 = build_4x4_network(seed=42)
    print("\n  Network topology (random weights, seed=42):")
    for u, v, c in edges_4x4:
        print(f"    {u:>2} -- {v:>2} : cost {c}")
    iters_4x4 = run_dv(nodes_4x4, edges_4x4, label="Task 1-2")

    # ---- Summary ----
    print("\n\n" + "=" * 70)
    print("  SUMMARY")
    print("=" * 70)
    print(f"  Task 1-1 (2x2): Converged in {iters_2x2} iteration(s)")
    print(f"  Task 1-2 (4x4): Converged in {iters_4x4} iteration(s)")
```

```python


if __name__ == "__main__":
    main()
```

Listing 4: DV Routing Algorithm Implementation

Lab 2: Protection Strategies (protection.py)

```python
#!/usr/bin/env python3
"""
Protection Strategies: CRC-8 and SEC-DED Implementation
HW6 - Computer Communications & Networks
Author: Yousef Alaa Awad

Implements:
  Task 2-1: CRC-8 (polynomial 0x107) and SEC-DED encoding for a 64-bit input.
  Task 2-2: Error injection & detection rate evaluation (1,000,000 trials each).
"""

import random

# ============================================================================
# Constants
# ============================================================================

# The fixed 64-bit input value
INPUT_VALUE = 0xFA7816EB87CC0911
INPUT_BITS = 64

# CRC polynomial: x^8 + x^2 + x + 1 = 0x107 (9 bits, degree 8)
CRC_POLY = 0x107
CRC_BITS = 8

# Number of trials for error detection evaluation
NUM_TRIALS = 1_000_000


# ============================================================================
# CRC-8 Implementation
# ============================================================================

def crc8_compute(data, data_bits=64):
    """
    Compute CRC-8 remainder for the given data using polynomial 0x107.

    The CRC is computed by appending 8 zero bits to the data and then
    performing polynomial long division (XOR-based) in GF(2).

    Args:
        data: Integer representing the data bits.
        data_bits: Number of bits in the data.

    Returns:
        The 8-bit CRC remainder as an integer.
    """
    # Shift data left by CRC_BITS to append zeros for division
    augmented = data << CRC_BITS
    total_bits = data_bits + CRC_BITS

    # Perform XOR-based polynomial division from MSB to LSB
    for i in range(total_bits - 1, CRC_BITS - 1, -1):
        if augmented & (1 << i):  # If the current bit is 1
            augmented ^= CRC_POLY << (i - CRC_BITS)

    # The remainder (lower 8 bits) is the CRC
    return augmented & 0xFF


def crc8_encode(data, data_bits=64):
    """
    Encode data with CRC-8 by appending the CRC remainder.
    Returns (crc_value, encoded_data) where encoded_data = data || CRC.
    """
```

```python
    crc = crc8_compute(data, data_bits)
    encoded = (data << CRC_BITS) | crc
    return crc, encoded


def crc8_check(received, total_bits=72):
    """
    Check if received data (data + CRC) has errors.
    Divides the entire received message by the polynomial.
    If the remainder is 0, no error is detected.

    Returns:
        True if error is detected (remainder != 0), False otherwise.
    """
    # Divide the received value (which already includes the CRC) by the polynomial.
    # No zero-appending needed -- the CRC bits are already in place.
    value = received
    for i in range(total_bits - 1, CRC_BITS - 1, -1):
        if value & (1 << i):
            value ^= CRC_POLY << (i - CRC_BITS)
    remainder = value & 0xFF
    return remainder != 0


# ==========================================================================
# SEC-DED (Hamming Code) Implementation
# ==========================================================================

def secded_compute_parity_positions(data_bits):
    """
    Determine the number of parity bits needed for SEC-DED.
    For SEC-DED, we need r parity bits such that 2^r >= data_bits + r + 1,
    plus 1 extra overall parity bit for double error detection.

    For 64 data bits: r = 7 (since 2^7 = 128 >= 64 + 7 + 1 = 72),
    plus 1 overall parity = 8 total parity bits => 72 total bits.
    """
    r = 0
    while (1 << r) < data_bits + r + 1:
        r += 1
    return r


def secded_encode(data, data_bits=64):
    """
    Encode data using SEC-DED (Hamming code + overall parity).

    Parity bits are placed at positions that are powers of 2 (1-indexed):
    positions 1, 2, 4, 8, 16, 32, 64. Data bits fill the remaining positions.
    An overall parity bit is placed at position 0 for double error detection.

    Args:
        data: Integer representing the data.
        data_bits: Number of data bits.

    Returns:
        (encoded_value, total_bits): The encoded integer and its total bit count.
    """
    r = secded_compute_parity_positions(data_bits)
    total = data_bits + r + 1  # +1 for overall parity bit at position 0

    # Build the codeword as a list (1-indexed positions)
    # Position 0 = overall parity, positions 1..total-1 = Hamming positions
    codeword = [0] * total

    # Place data bits into non-parity positions (1-indexed)
    # Parity positions are powers of 2: 1, 2, 4, 8, 16, 32, 64
    data_idx = data_bits - 1  # MSB first
```

```python
    for pos in range(1, total):
        if (pos & (pos - 1)) != 0:  # Not a power of 2 => data position
            if data_idx >= 0:
                codeword[pos] = (data >> data_idx) & 1
                data_idx -= 1

    # Compute each parity bit (covers positions where bit i of position is set)
    for i in range(r):
        parity_pos = 1 << i
        parity = 0
        for pos in range(1, total):
            if pos & parity_pos:
                parity ^= codeword[pos]
        codeword[parity_pos] = parity

    # Compute overall parity (position 0) = XOR of all bits
    codeword[0] = 0
    for pos in range(1, total):
        codeword[0] ^= codeword[pos]

    # Convert codeword list to integer (position 0 = MSB)
    encoded = 0
    for bit in codeword:
        encoded = (encoded << 1) | bit

    return encoded, total


def secded_check(received, total_bits=72):
    """
    Check received SEC-DED codeword for errors.

    Computes the syndrome (XOR of parity checks) and overall parity.
    - Syndrome == 0, overall parity OK => no error
    - Syndrome != 0, overall parity error => single-bit error (correctable)
    - Syndrome != 0, overall parity OK => double-bit error (detectable only)
    - Syndrome == 0, overall parity error => error in overall parity bit

    Returns:
        True if error is detected, False otherwise.
    """
    data_bits = 64
    r = secded_compute_parity_positions(data_bits)

    # Convert integer to bit list (MSB first, position 0 = MSB)
    bits = []
    for i in range(total_bits - 1, -1, -1):
        bits.append((received >> i) & 1)

    # Compute overall parity of all bits
    overall_parity = 0
    for b in bits:
        overall_parity ^= b

    # Compute syndrome from parity bit positions
    syndrome = 0
    for i in range(r):
        parity_pos = 1 << i
        parity = 0
        for pos in range(1, total_bits):
            if pos & parity_pos:
                parity ^= bits[pos]
        if parity != 0:
            syndrome |= (1 << i)

    # Detection logic:
    # If syndrome != 0 => error detected (single or double)
    # If syndrome == 0 and overall parity != 0 => error in overall parity bit
```

```python
    # If syndrome == 0 and overall parity == 0 => no error
    if syndrome != 0:
        return True  # Error detected
    if overall_parity != 0:
        return True  # Error in overall parity bit
    return False  # No error detected


# ========================================================================
# Error Injection Functions
# ========================================================================

def inject_single_bit_error(value, num_bits):
    """Flip exactly 1 random bit."""
    pos = random.randint(0, num_bits - 1)
    return value ^ (1 << pos)


def inject_two_bit_error(value, num_bits):
    """Flip exactly 2 random distinct bits."""
    positions = random.sample(range(num_bits), 2)
    for p in positions:
        value ^= (1 << p)
    return value


def inject_three_bit_error(value, num_bits):
    """Flip exactly 3 random distinct bits."""
    positions = random.sample(range(num_bits), 3)
    for p in positions:
        value ^= (1 << p)
    return value


def inject_burst_error(value, num_bits, burst_len):
    """
    Inject an n-bit burst error.
    The first and last bits of the burst window are always flipped.
    Intermediate bits are flipped with 50% probability.
    The burst start position is chosen randomly.
    """
    # Choose a random starting position for the burst
    start = random.randint(0, num_bits - burst_len)
    # First bit of burst is always flipped
    value ^= (1 << start)
    # Last bit of burst is always flipped
    value ^= (1 << (start + burst_len - 1))
    # Intermediate bits flipped with 50% probability
    for i in range(1, burst_len - 1):
        if random.random() < 0.5:
            value ^= (1 << (start + i))
    return value


# ========================================================================
# Task 2-1: Compute and display CRC and SEC-DED outputs
# ========================================================================

def task2_1():
    """Compute and display CRC-8 and SEC-DED encoding for the fixed input."""
    print("=" * 70)
    print("  TASK 2-1: CRC-8 and SEC-DED Encoding")
    print("=" * 70)
    print(f"\n  Input value: 0x{INPUT_VALUE:016X}")
    print(f"  Input (binary): {INPUT_VALUE:064b}")

    # CRC-8
    crc, crc_encoded = crc8_encode(INPUT_VALUE, INPUT_BITS)
```

```python
    print(f"\n  --- CRC-8 (Polynomial 0x107: x^8 + x^2 + x + 1) ---")
    print(f"  CRC-8 value:    0x{crc:02X} (binary: {crc:08b})")
    print(f"  Encoded data:   0x{crc_encoded:018X}")
    print(f"  Encoded (bin):  {crc_encoded:072b}")

    # Verify CRC: checking the encoded value should yield remainder 0
    has_error = crc8_check(crc_encoded, INPUT_BITS + CRC_BITS)
    print(f"  Verification:   {'ERROR' if has_error else 'OK (remainder = 0)'}")

    # SEC-DED
    secded_encoded, secded_total = secded_encode(INPUT_VALUE, INPUT_BITS)
    print(f"\n  --- SEC-DED (Hamming + overall parity) ---")
    print(f"  Total bits:     {secded_total} ({INPUT_BITS} data + "
          f"{secded_total - INPUT_BITS} parity)")
    print(f"  Encoded value:  0x{secded_encoded:018X}")
    print(f"  Encoded (bin):  {secded_encoded:0{secded_total}b}")

    # Verify SEC-DED: checking the encoded value should detect no error
    has_error = secded_check(secded_encoded, secded_total)
    print(f"  Verification:   {'ERROR' if has_error else 'OK (no error detected)'}")

    return crc_encoded, secded_encoded, secded_total


# ======================================================================
# Task 2-2: Error detection evaluation
# ======================================================================

def task2_2(crc_encoded, secded_encoded, secded_total):
    """
    Evaluate error detection rates for CRC-8 and SEC-DED.
    For each of 5 error types, inject errors into 1,000,000 random trials
    and record how often each scheme detects the error.
    """
    print("\n\n" + "=" * 70)
    print("  TASK 2-2: Error Detection Evaluation (1,000,000 trials each)")
    print("=" * 70)

    crc_total_bits = INPUT_BITS + CRC_BITS  # 72 bits for CRC-encoded data

    # Define the 5 error types as (name, injection_function)
    error_types = [
        ("Single-bit error",
         lambda v, n: inject_single_bit_error(v, n)),
        ("Two random-bit errors",
         lambda v, n: inject_two_bit_error(v, n)),
        ("Three random-bit errors",
         lambda v, n: inject_three_bit_error(v, n)),
        ("8-bit burst error",
         lambda v, n: inject_burst_error(v, n, 8)),
        ("16-bit burst error",
         lambda v, n: inject_burst_error(v, n, 16)),
    ]

    results = []

    for name, inject_fn in error_types:
        crc_detected = 0
        secded_detected = 0

        for _ in range(NUM_TRIALS):
            # Inject error into CRC-encoded value and check
            corrupted_crc = inject_fn(crc_encoded, crc_total_bits)
            if crc8_check(corrupted_crc, crc_total_bits):
                crc_detected += 1

            # Inject error into SEC-DED-encoded value and check
            corrupted_secded = inject_fn(secded_encoded, secded_total)
```

```python
            if secded_check(corrupted_secded, secded_total):
                secded_detected += 1

        crc_rate = (crc_detected / NUM_TRIALS) * 100
        secded_rate = (secded_detected / NUM_TRIALS) * 100
        results.append((name, crc_rate, secded_rate))

        print(f"\n  {name}:")
        print(f"    CRC-8 detection rate:   {crc_rate:.4f}%"
              f"  ({crc_detected}/{NUM_TRIALS})")
        print(f"    SEC-DED detection rate:  {secded_rate:.4f}%"
              f"  ({secded_detected}/{NUM_TRIALS})")

    # Print summary table
    print("\n\n  " + "-" * 66)
    print(f"  {'Error Type':<28} {'CRC Rate (%)':>15} {'SEC-DED Rate (%)':>18}")
    print("  " + "-" * 66)
    for name, crc_rate, secded_rate in results:
        print(f"  {name:<28} {crc_rate:>14.4f}% {secded_rate:>17.4f}%")
    print("  " + "-" * 66)


# ==========================================================================
# Main
# ==========================================================================

def main():
    crc_encoded, secded_encoded, secded_total = task2_1()
    task2_2(crc_encoded, secded_encoded, secded_total)


if __name__ == "__main__":
    main()
```

Listing 5: CRC-8 and SEC-DED Implementation